BranchBox: Stage and Target Supervision for Multi-Object Placement

Qinzhen Ma[1*] Harry Yang[2*] Changsheng Lu[2*] Xiaogang Xu[3]
[1]Rice University [2]The Hong Kong University of Science and Technology
[3]Zhejiang University
[*]Equal contribution.

Research preprint, November 2026.

Abstract

Multi-object manipulation requires a robot to switch both its active subtask and its spatial target. We study structured supervision with BranchBox, a vision-language-action training recipe combining stage-semantic instructions, a separate marked image, and auxiliary stage and target heads on action-decoder features. Prompt dropout and temporal perturbations support ordinary test inputs or prediction feedback through a fixed task-specific stage table. In a post hoc exploratory analysis of four two-object LIBERO-Long tasks, SmolVLA and X-VLA use matched training data and paired initial states. With self-generated prompts, success reaches 56.9% versus 45.6% for matched action-only training: an 11.3-percentage-point gain (unadjusted 95% CI: 1.9 to 20.6) and a 24.7% relative increase. Both models have positive point estimates; ordinary test inputs yield 55.6% success. A separate, fully fine-tuned memory extension does not improve the existing X-VLA policy on a 20-scene development panel. Complete broader-study results and memory diagnostics accompany the analysis. These conditional findings motivate stage and target supervision for multi-object placement, with generality still to be tested beyond the retrospectively selected tasks and models.

1. Introduction

Placing two objects requires more than repeating a single grasp. After completing one placement, the robot must select the next object, shift its spatial target, and continue toward the remaining goal. A task instruction specifies what should be achieved, but a vision-language-action (VLA) policy must infer this changing stage and target from its observations [3, 12, 20, 28]. We ask whether a compact, supervised stage/target interface can improve these routines when no additional learned planner runs online.

Intermediate representations are already useful interfaces for robot learning. RT-H uses language motions [2], ECoT predicts semantic and visually grounded quantities [25], VP-VLA conditions a controller on stage commands and marked images [21], and GuidedVLA supervises task-relevant factors in action-decoder attention [11]. We build on this line of work with a compact implementation for lightweight policies: two small heads predict a stage and spatial target from action-decoder features, while training also exposes the controller to stage text and visual marks.

BranchBox uses a separate prompt-image slot so that the original camera views remain available. Text and marks are independently dropped and temporally perturbed during training. The trained network can then act from ordinary observations, or return its previous predictions through the text and image interface. A fixed task-specific table maps the predicted stage to its instruction and mark type; deployment does not require an additional learned planner or tracker. This table supplies task structure, while the policy predicts where it is within that structure.

We study four two-object placement tasks from our evaluated LIBERO-Long set [15]: two moka pots, two mugs, a mug and pudding, and soup and sauce. In the focused exploratory analysis, success increases from 45.6% with matched action-only training to 56.9% with BranchBox and self-generated prompts: an absolute gain of 11.3 percentage points and a 24.7% relative increase. Both model families and all four tasks have positive point estimates. The same trained networks reach 55.6% with ordinary inputs, supporting deployment when test-time prompting is undesirable.

The delayed stage/target interface carries only a compact summary into the next query. A separate experiment asks whether an already trained X-VLA policy can use richer interaction history. We adapt MemBodied [17], preserving its associative state, dedicated memory token, episode-first anchor and execution-conditioned write. We compare the final

fine-tuned model with the existing policy and with the same final weights whose added memory paths are reset between queries, on a fixed 20-scene development panel.

Contributions. We present (1) a structured training recipe that supervises stage and spatial-target prediction on action-decoder features while perturbing the corresponding input prompts; (2) a deployment interface that supports ordinary observations or self-generated prompts through a supplied task-specific stage table; and (3) paired empirical studies of the recipe in two lightweight policies and of added history use in the trained X-VLA policy. The original four-task analysis identifies positive point estimates relative to matched native training. The additional same-weight intervention tests history use after full fine-tuning. The complete experimental scope and comparisons are reported in the supplementary material.

2. Related Work

Visual prompts at the policy input. Marks rendered into an image let multimodal models refer to regions: Set-of-Mark labels segments for GPT-4V [23], MOKA pairs task decomposition with marked keypoints for manipulation [5], and PIVOT iteratively draws candidate actions for a vision-language model to select [16]. TraceVLA [29] uses motion traces from a point tracker. Policies conditioned on drawn trajectories (RT-Trajectory [6], HAMSTER [13]) or pretrained with mark supervision (Magma [24]) learn to read the cue during training. VP-VLA [21] uses a Qwen3-VL planner [1] and SAM3 grounding to construct stage instructions and visual prompts. Its auxiliary grounding objective predicts discretized coordinates at key frames using cross-entropy, with grounding gradients updating the VLM. We retain its separate prompt-image interface, while stage classification and continuous coordinate regression supervise the action decoder with the visual/language backbones frozen in the original two-model study.

Hierarchical language conditioning. SayCan combines a language model's estimate of skill relevance with learned affordances [10], and Inner Monologue feeds success detection and scene descriptions back to a language planner [9]. RT-H trains a policy to predict intermediate language motions before actions [2]; YAY Robot uses human language corrections to a high-level policy [18]; Hi Robot trains a vision-language model to issue commands to a VLA [19]; and $\pi_{0.5}$ predicts a semantic subtask before its actions [3]. BranchBox shares the use of an explicit intermediate representation, and pairs a stage with a spatial target, inside one lightweight network and with no second model at test time.

Policy-generated intermediate representations. ECoT [25] trains a VLA to generate task decompositions and visually grounded quantities, including object boxes, before predicting actions. CoT-VLA [27] predicts a future image as an intermediate visual goal, while DiffusionVLA [22] injects self-generated language reasoning into its action model. SPR [4] predicts remaining spatial subtasks and gripper trajectories, and monitors progress to trigger a learned rewind behavior. These methods predict structured intermediate representations within the policy. GuidedVLA [11] supervises designated action-decoder attention heads for object and skill factors, with depth features and residual attention fusion. BranchBox instead uses small heads on pooled action-decoder features to predict a stage index and spatial target. A supplied task-specific stage table converts these predictions into delayed text and image prompts. This avoids autoregressive generation of a reasoning sequence or goal image, but does not provide open-ended task decomposition. We evaluate the training recipe separately from the effect of feeding its predictions back into the policy.

Lightweight VLAs. We instantiate the recipe in SmolVLA [20] and X-VLA [28], two compact policies with different action decoders. The original two-model recipe trains the decoder and its auxiliary heads while freezing the visual and language backbones. The architecture also includes a prompt-image slot; we do not claim a measured system-cost advantage over the above methods. The separate memory extension uses full-weight fine-tuning.

Interventions around frozen policies. BYOVLA edits task-irrelevant image regions at run time [8], RoboHarness adds episodic memory and failure attribution [14], and Harness VLA composes a frozen VLA with analytic primitives [26]. These methods leave the weights alone. Our work instead studies structured supervision during policy training.

Recurrent policy memory. MemBodied [17] combines a recurrent associative state with an initial-scene anchor. SmoLSTM [7] uses persistent matrix-LSTM state and evaluates the trained policy with its state reset between decisions. Our additional X-VLA study adopts MemBodied's core mechanisms and uses a same-weight reset intervention to test reliance on history. Its contrast with the existing checkpoint also includes additional full-weight fine-tuning; it is not an isolated comparison of memory architectures or a reproduction of the papers' published benchmark scores.

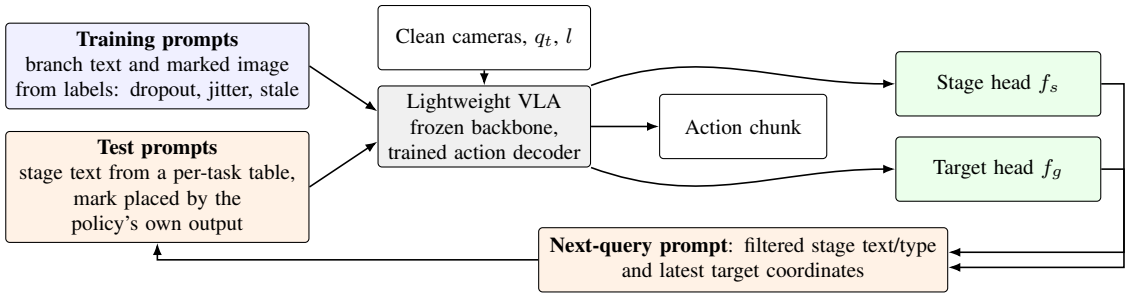


Figure 1. BranchBox in the original two-model study. In training, labels supply the prompts, and the two heads learn the current stage and target. At test time the previous query's predictions supply the next prompt through a fixed task table. Stage predictions are filtered; target coordinates are taken from the latest query. No external planner, tracker or simulator state supplies these predictions.

3. Method

Task interface and action policy. At query t, a policy receives camera images I_t, proprioception q_t and instruction l, and predicts an action chunk A_t. The first h actions are executed before the next query. Each task has an ordered table $\mathcal{T} = \{(s_k, m_k)\}_{k=0}^{K-1}$, where s_k is stage text and m_k is a crosshair, box, or no mark. We fix the table from demonstration annotation metadata before training and supply it at test time. The policy predicts progress within this supplied decomposition. Our implementation supports at most six stages.

Training prompts and supervision. The branch instruction appends `Current subtask:` and s_{k_t} to l. Following VP-VLA [21], the main camera stays clean and a marked copy enters a separate image slot; the wrist camera is not marked. A crosshair denotes an object to reach, and a box denotes a destination or part. Offline simulator predicates, contacts and segmentation provide training-stage and normalized image-target annotations. For the two-object tasks, k_t is twice the subtask position plus a reach/place phase, giving four stages (Tab. 1). The two phases share a subtask text but use different mark types. The supplementary material specifies the annotation rules; the test-time prompter never queries these labels.

Prompt perturbations. Text and marks are independently retained with probability 0.5. Mark coordinates are independently perturbed by up to 5% of the image extent and clipped to $[0, 1]$. At demonstration frame u, with probability 0.5, prompt labels come from frame $u - L$, with L uniform on $1, \ldots, 3h$. Otherwise, a frame offset in $\{-5, \ldots, -1, 1, \ldots, 5\}$ is applied with probability 0.5. Indices are clipped to the current demonstration. Labels from the selected frame are rendered onto the current image, while auxiliary supervision always uses the current frame. Draws are deterministic functions of the seed, sample index and epoch. These perturbations expose the network to missing and delayed cues while retaining current-frame supervision.

Auxiliary prediction. Two MLPs, each with two 256-unit ReLU hidden layers, read the input features to the action projection, averaged over action-chunk tokens. The supplementary material gives the model-specific attachment points. The stage head has six logits; the target head outputs $(p_x, p_y, x_0, y_0, x_1, y_1)$ through a sigmoid in normalized image coordinates. The loss is

$$\mathcal{L} = \mathcal{L}_{\text{action}} + 0.1\mathcal{L}_{\text{stage}} + 0.1\mathcal{L}_{\text{target}}. \tag{1}$$

Stage loss is six-way cross-entropy on labeled samples; task-specific class restriction is used only at deployment. Target loss applies L1 supervision only to valid coordinates of the annotated mark type: point coordinates for crosshairs and box coordinates for boxes. Within each valid sample the absolute errors are averaged over its two or four supervised coordinates, then averaged over valid samples; a batch with no valid targets has zero target loss. Thus, not all six target outputs are supervised on every frame. Both auxiliary weights are fixed at 0.1; the objectives are stage cross-entropy and continuous target regression, distinct from VP-VLA's grounding objective. In the original two-model recipe, the visual and language backbones remain frozen, while auxiliary gradients update the action-decoder representation. Training features come from the training forward pass, while inference features are taken from the final action-generation step.

Feedback at test time. The first query uses stage-0 text and a clean prompt image. After each new chunk, we take the stage argmax only over the K stages valid for the task ($K = 4$ in the focused tasks). The filtered stage never decreases and advances after two consecutive predictions agree on a later stage; jumps over intermediate stages are allowed. The next query uses the filtered stage's text and mark type, drawing the latest target coordinates on the new

camera image without tracking or geometric warping. Only observations, policy outputs and the fixed task table enter this loop.

The filter and target are updated on different rules; the supplementary material details the resulting alignment and recovery limitations. The two-model results use this recorded implementation.

Persistent-history extension. We additionally study whether a trained X-VLA policy benefits from preserving interaction history. We adapt the four core mechanisms of MemBodied [17]: associative reads and transition-conditioned writes in all 24 action-expert layers, dedicated state/memory tokens, and an episode anchor that conditions the action input. A write uses the previous chunk's actually executed controls and the next observation; memory remains fixed within action denoising. The original first-frame images supply the anchor through X-VLA's native visual encoder. The supplementary material specifies this backbone mapping, the equations, full-weight fine-tuning, and same-weight history-reset comparison.

4. Evaluation Protocol

We distinguish the original two-model training-recipe study from an additional full-weight memory adaptation of its trained X-VLA policy. Their checkpoints, initial-state panels and contrasts are separate.

Focused task family. Among the six evaluated LIBERO-Long tasks, four require placing two objects. We examine these four tasks and two model families as a retrospectively selected exploratory subset. The tasks share a four-stage reach/place structure (Tab. 1, Fig. 2). The complete broader study, including the remaining task types, model configurations, original primary analysis and protocol deviations, is retained in the supplementary material. This focus is not a prespecified endpoint or a benchmark-wide performance claim.

Original two-model training. Training uses 203 demonstrations and 58,272 frames from all six evaluated LIBERO tasks, including the two tasks outside the focused analysis. Thus, the present grouping changes the analysis scope, not the trained checkpoints or their data. SmolVLA [20] starts from `HuggingFaceVLA/smolvla_libero` and trains its action expert and state/action projections for 14,000 steps. X-VLA [28] starts from `lerobot/xvla-base` and trains its transformer and soft prompts for 20,000 steps. Batch size is 32 and the training seed is 1. Native and BranchBox runs share initial weights, sample order, update counts and seed; BranchBox adds prompt exposure and auxiliary supervision. We evaluate only final checkpoints.

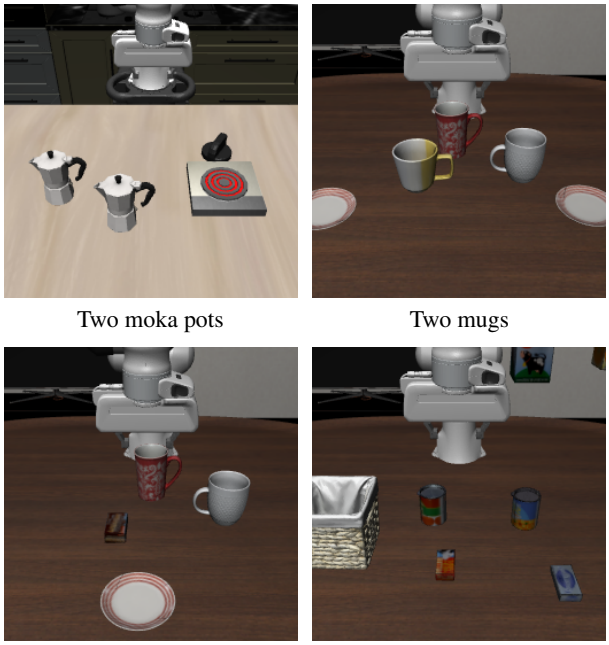


Figure 2. The four two-object placement routines. These unmodified main-camera observations show official initial-state 40 from the separate development panel, before any policy action. The same fixed state index was chosen for each task before memory evaluation; these are task illustrations, not selected success trajectories.

The supplementary material records demonstration filtering, label construction, adapter settings, and feedback pseudocode. Demonstrations are replayed from their recorded initial states; evaluation uses the official initial-state list.

Matched evaluation arms. N0 is the matched action-only baseline with the plain instruction and a clean main-camera copy in the prompt slot. P0 is the BranchBox-trained network with the same inputs. PS uses that network with its own generated prompts. All arms contain the same image slots and head modules; N0 auxiliary heads have zero training weight. N0 therefore controls the recipe within this shared architecture; it is not the untouched official checkpoint or its published benchmark score. Comparisons against N0 measure the combined training recipe under each deployment mode. P0 uses no external prompt source, and PS uses only policy predictions and its fixed task table.

Original paired evaluation. Each model–task–arm runs official initial states 20–39 with matched episode seeds. The four tasks provide 80 distinct task–initial-state pairs, each evaluated by both models: 160 model–scene evaluations per arm and 480 episodes in total. Initial-frame hashes and

Task (ID)	First subtask	Second subtask
Moka pots (8)	Second pot → stove	First pot → stove
Two mugs (4)	White mug → left plate	Yellow/white mug → right plate
Mug/pudding (6)	White mug → plate	Pudding → right of plate
Soup/sauce (0)	Alphabet soup → basket	Tomato sauce → basket

Table 1. Supplied decompositions for the focused LIBERO-Long tasks (zero-based `libero_10` IDs). Each subtask uses two stages with the same text: a crosshair on its object, then a box on its destination, giving four stages per task. Exact strings are in the supplementary material.

episode seeds match across all six model–arm combinations on every focused scene. Images are 256×256; ten settling actions precede a 520-step horizon. SmolVLA executes 10 actions per query and X-VLA executes 30.

For each bootstrap replicate, we independently resample the 20 initial states within each task and share those indices across *both models and all arms*. We then average equally over tasks and models. This preserves their common-scene dependence. We use $B = 100{,}000$ replicates, seed 20261101, and percentile intervals. Intervals condition on the fixed checkpoints and tasks; they are unadjusted for retrospective subset selection and multiple contrasts. The supplementary material gives the exact task order, a resampling sensitivity check, and complete broader-study results. The source package includes per-episode success indicators and an executable analysis script.

Separate memory-development panel. The memory experiment starts from the existing BranchBox-trained X-VLA-P checkpoint, which is distinct from N0. It adds 6,000 full-weight updates on 160 demonstrations; 43 demonstrations monitor training but do not select the checkpoint. Master seed is one. The final EMA, the existing checkpoint, and the same final weights with both added memory paths reset are evaluated on official initial states 40–44 for each focused task: 20 paired scenes and 60 rollouts. These scenes were used in earlier pilots. Reset clears both recurrent state and the initial-scene anchor at every query; the original BranchBox stage filter and prompt feedback remain active in all three conditions. The existing-checkpoint contrast includes additional fine-tuning. Reset is a test-time intervention on a policy trained with persistent history: it tests that policy's historical-information use, not superiority over an independently trained memory-free policy. Exact demonstration–evaluation initial-state overlap was not verified. Intervals resample paired scenes within task; exact McNemar tests use Holm adjustment over the two contrasts. The supplementary material provides the full architecture, continuation schedule, control definition and per-scene outcomes.

5. Results on Multi-Object Placement

An 11.3-point absolute gain and 24.7% relative increase. Across the four tasks and two models, PS solves 91 of 160 episodes compared with 73 for N0: 56.9% versus 45.6%, a difference of $+11.3$ percentage points (unadjusted 95% CI $+1.9$ to $+20.6$; Tab. 2). Among discordant pairs, PS succeeds on 38 model–scene pairs where N0 fails, versus 20 in the opposite direction. This corresponds to a 24.7% relative increase, computed from the unrounded success counts. This is the same effect expressed relative to native success, within the selected multi-object task family.

Positive point estimates in both model families. SmolVLA increases from 42/80 to 50/80 successes (52.5% to 62.5%; $+10.0$ points). X-VLA increases from 31/80 to 41/80 (38.8% to 51.3%; $+12.5$ points). Table 2 reports the wider per-model intervals. The common direction across the two architectures motivates further tests of the same task structure with independently trained policies.

A 10-point gain using ordinary test inputs. P0 solves 89/160 episodes (55.6%), compared with 73/160 for N0, for $+10.0$ points (unadjusted 95% CI 0.0 to $+19.4$). Its model-level differences are $+11.3$ points for SmolVLA and $+8.8$ for X-VLA. This deployment option retains the trained action policy while using a plain instruction and clean images at test time. The comparison measures the full training recipe, including both auxiliary supervision and prompt exposure. P0 and PS are deployment choices for the same trained policy; their direct comparison appears in the supplementary material.

Gains span four distinct two-object routines. All four tasks have positive PS$-$N0 and P0$-$N0 pooled point estimates (Tab. 3). With self-generated prompts, the differences are $+10.0$ points on moka pots, $+15.0$ on two mugs, $+7.5$ on mug and pudding, and $+12.5$ on soup and sauce. The largest P0 difference occurs on moka pots: 33/40 versus 25/40, or $+20.0$ points. Each deployment arm is reported separately rather than choosing the better arm for each task. These per-task estimates describe variation within the selected family; their uncertainty is shown in Fig. 3.

Interpreting the task structure. Each focused task requires two reach/place cycles and a change of object identity between them. Stage and target supervision gives the action decoder explicit labels for these intermediate variables. The observed pattern is compatible with this motivation, but successful stage transitions and target switch-

Model	N0 matched	P0 ordinary input	PS self-prompt	P0−N0 (points)	PS−N0 (points)
SmolVLA	42/80 (52.5%)	51/80 (63.8%)	50/80 (62.5%)	+11.3 [-1.3, +23.8]	+10.0 [-2.5, +22.5]
X-VLA	31/80 (38.8%)	38/80 (47.5%)	41/80 (51.3%)	+8.8 [-3.8, +21.3]	+12.5 [-1.3, +26.3]
Both models	73/160 (45.6%)	89/160 (55.6%)	91/160 (56.9%)	+10.0 [0.0, +19.4]	+11.3 [+1.9, +20.6]

Table 2. Four two-object LIBERO placement tasks. The two models share 80 task–initial-state pairs. Brackets give unadjusted 95% paired bootstrap intervals, with scene indices shared across models and arms. The subset was selected retrospectively; complete broader-study comparisons are in the supplementary material.

Task	N0	P0	PS	P0−N0	PS−N0
Moka pots	25/40	33/40	29/40	+20.0	+10.0
Two mugs	15/40	16/40	21/40	+2.5	+15.0
Mug and pudding	18/40	22/40	21/40	+10.0	+7.5
Soup and sauce	15/40	18/40	20/40	+7.5	+12.5

Table 3. Per-task success pooled over both models. Differences are percentage points; paired uncertainty is plotted in Fig. 3.

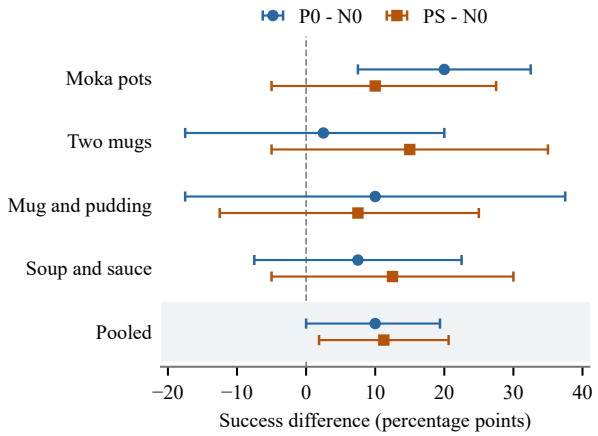


Figure 3. Differences against N0 on the selected two-object tasks. Points are observed effects; bars are unadjusted 95% scene-clustered paired bootstrap intervals. P0 and PS are fixed deployment modes, each evaluated on every task.

ing were not measured as causal mediators. Their contribution requires dedicated diagnostics rather than inference from task success alone.

5.1. Persistent history in X-VLA

On the separate 20-scene development panel, the final memory model solves 12/20 episodes, compared with 13/20 for the existing X-VLA-P and 11/20 with the same weights and both added memory paths reset (Tab. 4).

Relative to the existing checkpoint, the paired difference is -5.0 percentage points (pointwise 95% CI $[-30.0, +20.0]$; 4 rescued scenes and 5 regressions; Holm-adjusted exact McNemar $p = 1.000$).

Condition	Successes	Success rate
Existing X-VLA-P	13/20	65.0%
Full memory, final EMA	12/20	60.0%
Same weights, added-memory reset	11/20	55.0%

Table 4. Separate X-VLA-P memory-development panel: 20 paired scenes, master seed one, 6,000 additional full-weight updates. The existing policy is the BranchBox-trained checkpoint, not N0. The reset condition uses exactly the full-memory weights while resetting the matrix and first-frame anchor at every query. The original BranchBox prompt controller remains active in all three conditions.

Relative to the same-weight reset, the paired difference is $+5.0$ percentage points (pointwise 95% CI $[-15.0, +25.0]$; 3 rescued scenes and 2 regressions; Holm-adjusted exact McNemar $p = 1.000$).

The existing-checkpoint comparison includes additional full-weight training. The same-weight intervention measures the combined use of recurrent and initial-scene history; it does not isolate the anchor or compare against an equally fine-tuned memory-free model. These are conditional development-panel estimates from one training run. All paired outcomes and task-level results appear in the supplementary material.

6. Scope and Next Steps

The analysis concerns a task and model subset chosen after observing a broader experiment. Its intervals are conditional on one training seed and are not selection-adjusted, so the numerical gains should be interpreted as exploratory evidence for this task family. Independent seed pairs and independently selected multi-object tasks are the next tests of generality. The supplementary material preserves the complete original experimental scope, comparisons between deployment modes and implementation diagnostics.

The training recipe combines input prompts and auxiliary supervision; a heads-only training control would help identify their individual roles. The current interface also requires a supplied stage table and uses delayed predictions. Stage-consistent target feedback and recovery from incorrect stage transitions are concrete extensions to evaluate.

These are future evaluation conditions, not changes underlying the reported results.

The additional memory comparison uses one trained model and 20 scenes that appeared in earlier development. Reset jointly removes its added recurrent and episode-anchor histories while retaining the original prompt controller. A matched additional-training control and separate anchor/recurrent interventions would be needed to distinguish these effects. Neither study establishes transfer to new task decompositions or real robots.

7. Conclusion

BranchBox combines stage and spatial-target supervision on action-decoder features. On a post hoc subset of four two-object LIBERO placement tasks, success rises from 45.6% to 56.9% with self-generated prompts (an absolute gain of 11.3 points and a 24.7% relative increase), and reaches 55.6% with ordinary test inputs. Both model families and all four tasks have positive point estimates against matched native training. A separate 6,000-update memory adaptation solves 12/20 development scenes, versus 13/20 for the existing X-VLA policy and 11/20 under same-weight history reset; it does not establish an overall improvement. The positive evidence concerns the structured training recipe, with independent replication needed to establish its generality.

LLM Usage Statement

LLM-based assistants (OpenAI Codex and Anthropic Claude Code) supported research planning and literature retrieval, software implementation, debugging and testing, experiment operation on the cluster, analysis scripting, figure layout and plotting code, and manuscript drafting and editing under author direction. Experimental numbers were computed from the recorded episode and training logs. The authors are responsible for all claims.

References

[1] Shuai Bai, Yuxuan Cai, Ruizhe Chen, Keqin Chen, Xionghui Chen, Zesen Cheng, Lianghao Deng, Wei Ding, Chang Gao, Chunjiang Ge, Wenbin Ge, Zhifang Guo, Qidong Huang, Jie Huang, Fei Huang, Binyuan Hui, Shutong Jiang, Zhaohai Li, Mingsheng Li, Mei Li, Kaixin Li, Zicheng Lin, Junyang Lin, Xuejing Liu, Jiawei Liu, Chenglong Liu, Yang Liu, Dayiheng Liu, Shixuan Liu, Dunjie Lu, Ruilin Luo, Chenxu Lv, Rui Men, Lingchen Meng, Xuancheng Ren, Xingzhang Ren, Sibo Song, Yuchong Sun, Jun Tang, Jianhong Tu, Jianqiang Wan, Peng Wang, Pengfei Wang, Qiuyue Wang, Yuxuan Wang, Tianbao Xie, Yiheng Xu, Haiyang Xu, Jin Xu, Zhibo Yang, Mingkun Yang, Jianxin Yang, An Yang, Bowen Yu, Fei Zhang, Hang Zhang, Xi Zhang, Bo Zheng, Humen Zhong, Jingren Zhou, Fan Zhou, Jing Zhou, Yuanzhi Zhu, and Ke Zhu. Qwen3-VL technical report, 2025. arXiv:2511.21631. 2

[2] Suneel Belkhale, Tianli Ding, Ted Xiao, Pierre Sermanet, Quan Vuong, Jonathan Tompson, Yevgen Chebotar, Debidatta Dwibedi, and Dorsa Sadigh. RT-H: Action hierarchies using language. In *Proceedings of Robotics: Science and Systems (RSS)*, Delft, Netherlands, 2024. arXiv:2403.01823. 1, 2

[3] Kevin Black, Noah Brown, James Darpinian, Karan Dhabalia, Danny Driess, Adnan Esmail, Michael Robert Equi, Chelsea Finn, Niccolo Fusai, Manuel Y. Galliker, Dibya Ghosh, Lachy Groom, Karol Hausman, Brian Ichter, Szymon Jakubczak, Tim Jones, Liyiming Ke, Devin LeBlanc, Sergey Levine, Adrian Li-Bell, Mohith Mothukuri, Suraj Nair, Karl Pertsch, Allen Z. Ren, Lucy Xiaoyang Shi, Laura Smith, Jost Tobias Springenberg, Kyle Stachowicz, James Tanner, Quan Vuong, Homer Walke, Anna Walling, Haohuan Wang, Lili Yu, and Ury Zhilinsky. $\pi_{0.5}$: a vision-language-action model with open-world generalization. In *Proceedings of The 9th Conference on Robot Learning (CoRL)*, pages 17–40. PMLR, 2025. arXiv:2504.16054 (Physical Intelligence). 1, 2

[4] Tingjun Dai, Mingfei Han, Tingwen Du, Zhiheng Liu, Zihao Zhang, Zhihui Li, Salman Khan, Jun Yu, and Xiaojun Chang. See, plan, rewind: Progress-aware vision-language-action models for robust robotic manipulation, 2026. arXiv:2603.09292. 2

[5] Kuan Fang, Fangchen Liu, Pieter Abbeel, and Sergey Levine. MOKA: Open-world robotic manipulation through mark-based visual prompting. In *Robotics: Science and Systems*, 2024. 2

[6] Jiayuan Gu, Sean Kirmani, Paul Wohlhart, Yao Lu, Montserrat Gonzalez Arenas, Kanishka Rao, Wenhao Yu, Chuyuan Fu, Keerthana Gopalakrishnan, Zhuo Xu, Priya Sundaresan, Peng Xu, Hao Su, Karol Hausman, Chelsea Finn, Quan Vuong, and Ted Xiao. RT-Trajectory: Robotic task generalization via hindsight trajectory sketches. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2311.01977. 2

[7] Jan-Gerrit Habekost, Parsa Mastouri Kashani, Connor Gäde, Matthias Kerzel, Philipp Allgeuer, Cornelius Weber, Stefan Wermter, and Jae Hee Lee. SmoLSTM: A compact vision-language-action model with recurrent memory that persists, 2026. arXiv:2609.22854. 2

[8] Asher J. Hancock, Allen Z. Ren, and Anirudha Majumdar. Run-time observation interventions make vision-language-action models more visually robust. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 9499–9506, 2025. arXiv:2410.01971. 2

[9] Wenlong Huang, Fei Xia, Ted Xiao, Harris Chan, Jacky Liang, Pete Florence, Andy Zeng, Jonathan Tompson, Igor Mordatch, Yevgen Chebotar, Pierre Sermanet, Tomas Jackson, Noah Brown, Linda Luu, Sergey Levine, Karol Hausman, and Brian Ichter. Inner monologue: Embodied reasoning through planning with language models. In *Proceedings of The 6th Conference on Robot Learning (CoRL)*, pages 1769–1782. PMLR, 2023. arXiv:2207.05608. 2

[10] Brian Ichter, Anthony Brohan, Yevgen Chebotar, Chelsea Finn, Karol Hausman, Alexander Herzog, Daniel Ho, Julian Ibarz, Alex Irpan, Eric Jang, Ryan Julian, Dmitry Kalashnikov, Sergey Levine, Yao Lu, Carolina Parada, Kanishka Rao, Pierre Sermanet, Alexander T. Toshev, Vincent Vanhoucke, Fei Xia, Ted Xiao, Peng Xu, Mengyuan Yan, Noah Brown, Michael Ahn, Omar Cortes, Nicolas Sievers, Clayton Tan, Sichun Xu, Diego Reyes, Jarek Rettinghouse, Jornell Quiambao, Peter Pastor, Linda Luu, Kuang-Huei Lee, Yuheng Kuang, Sally Jesmonth, Nikhil J. Joshi, Kyle Jeffrey, Rosario Jauregui Ruano, Jasmine Hsu, Keerthana Gopalakrishnan, Byron David, Andy Zeng, and Chuyuan Kelly Fu. Do as I can, not as I say: Grounding language in robotic affordances. In *Proceedings of The 6th Conference on Robot Learning (CoRL)*, pages 287–318. PMLR, 2023. arXiv:2204.01691. 2

[11] Xiaosong Jia, Bowen Yang, Zuhao Ge, Xian Nie, Yuchen Zhou, Cunxin Fan, Yufeng Li, Yilin Chai, Chao Jing, Zijian Liang, Qingwen Bu, Haidong Cao, Chao Wu, Qifeng Li, Zhenjie Yang, Chenhe Zhang, Hongyang Li, Zuxuan Wu, Junchi Yan, and Yu-Gang Jiang. Guidedvla: Specifying task-relevant factors via plug-and-play action attention specialization. In *Proceedings of Robotics: Science and Systems*, 2026. 1, 2

[12] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan P Foster, Pannag R Sanketi, Quan Vuong, Thomas Kollar, Benjamin Burchfiel, Russ Tedrake, Dorsa Sadigh, Sergey Levine, Percy Liang, and Chelsea Finn. OpenVLA: An open-source vision-language-action model. In *Proceedings of The 8th Conference on Robot Learning (CoRL)*, pages 2679–2713. PMLR, 2025. arXiv:2406.09246. 1

[13] Yi Li, Yuquan Deng, Jesse Zhang, Joel Jang, Marius Memmel, Caelan Reed Garrett, Fabio Ramos, Dieter Fox, Anqi Li, Abhishek Gupta, and Ankit Goyal. HAMSTER: Hierarchical action models for open-world robot manipulation. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2502.05485. 2

[14] Zhuoran Li, Zhiyang Li, Kaijun Zhou, and Jinyu Gu. RoboHarness: A memory-augmented policy harness for vision-language-action model robustness via in-context adaptation, 2026. arXiv:2603.24060. 2

[15] Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. LIBERO: Benchmarking knowledge transfer for lifelong robot learning. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, pages 44776–44791, 2023. arXiv:2306.03310. 1

[16] Soroush Nasiriany, Fei Xia, Wenhao Yu, Ted Xiao, Jacky Liang, Ishita Dasgupta, Annie Xie, Danny Driess, Ayzaan Wahid, Zhuo Xu, Quan Vuong, Tingnan Zhang, Tsang-Wei Edward Lee, Kuang-Huei Lee, Peng Xu, Sean Kirmani, Yuke Zhu, Andy Zeng, Karol Hausman, Nicolas Heess, Chelsea Finn, Sergey Levine, and Brian Ichter. PIVOT: Iterative visual prompting elicits actionable knowledge for VLMs, 2024. arXiv:2402.07872. 2

[17] Tej Deep Pala, Navonil Majumder, Bryce Goh, Raphael Yee, Jianfei Yang, Liming Chen, and Soujanya Poria. MemBodied: Recurrent associative memory for vision-language-action models, 2026. arXiv:2609.28256. 1, 2, 4

[18] Lucy Xiaoyang Shi, Zheyuan Hu, Tony Z. Zhao, Archit Sharma, Karl Pertsch, Jianlan Luo, Sergey Levine, and Chelsea Finn. Yell at your robot: Improving on-the-fly from language corrections. In *Proceedings of Robotics: Science and Systems (RSS)*, Delft, Netherlands, 2024. arXiv:2403.12910. 2

[19] Lucy Xiaoyang Shi, Brian Ichter, Michael Robert Equi, Liyiming Ke, Karl Pertsch, Quan Vuong, James Tanner, Anna Walling, Haohuan Wang, Niccolo Fusai, Adrian Li-Bell, Danny Driess, Lachy Groom, Sergey Levine, and Chelsea Finn. Hi robot: Open-ended instruction following with hierarchical vision-language-action models. In *Proceedings of the 42nd International Conference on Machine Learning*, pages 54919–54933. PMLR, 2025. 2

[20] Mustafa Shukor, Dana Aubakirova, Francesco Capuano, Pepijn Kooijmans, Steven Palma, Adil Zouitine, Michel Aractingi, Caroline Pascal, Martino Russi, Andres Marafioti, Simon Alibert, Matthieu Cord, Thomas Wolf, and Remi Cadene. SmolVLA: A vision-language-action model for affordable and efficient robotics. arXiv:2506.01844, 2025. 1, 2, 4

[21] Zixuan Wang, Yuxin Chen, Yuqi Liu, Jinhui Ye, Pengguang Chen, Changsheng Lu, Shu Liu, Bei Yu, and Jiaya Jia. VP-VLA: Visual prompting as an interface for vision-language-action models, 2026. arXiv:2603.22003. 1, 2, 3

[22] Junjie Wen, Yichen Zhu, Minjie Zhu, Zhibin Tang, Jinming Li, Zhongyi Zhou, Xiaoyu Liu, Chaomin Shen, Yaxin Peng, and Feifei Feng. DiffusionVLA: Scaling robot foundation models via unified diffusion and autoregression. In *Proceedings of the 42nd International Conference on Machine Learning*, pages 66558–66574. PMLR, 2025. 2

[23] Jianwei Yang, Hao Zhang, Feng Li, Xueyan Zou, Chunyuan Li, and Jianfeng Gao. Set-of-mark prompting unleashes extraordinary visual grounding in GPT-4V, 2023. arXiv:2310.11441. 2

[24] Jianwei Yang, Reuben Tan, Qianhui Wu, Ruijie Zheng, Baolin Peng, Yongyuan Liang, Yu Gu, Mu Cai, Seonghyeon Ye, Joel Jang, Yuquan Deng, and Jianfeng Gao. Magma: A foundation model for multimodal AI agents. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 14203–14214, 2025. arXiv:2502.13130. 2

[25] Michał Zawalski, William Chen, Karl Pertsch, Oier Mees, Chelsea Finn, and Sergey Levine. Robotic control via embodied chain-of-thought reasoning. In *Proceedings of The 8th Conference on Robot Learning*, pages 3157–3181. PMLR, 2025. 1, 2

[26] Yixian Zhang, Huanming Zhang, Feng Gao, Xiao Li, Zhihao Liu, Yi Nie, Chunyang Zhu, Jiaxing Qiu, Yuchen Yan, Jiyuan Liu, Wenhao Tang, Jiaji Rao, Zhengru Fang, Changxu Wei, Yu Wang, Wenbo Ding, and Chao Yu. Harness VLA: Steering frozen VLAs into reliable manipulation primitives via memory-guided agents, 2026. arXiv:2607.08448. 2

[27] Qingqing Zhao, Yao Lu, Moo Jin Kim, Zipeng Fu, Zhuoyang Zhang, Yecheng Wu, Zhaoshuo Li, Qianli Ma, Song Han, Chelsea Finn, Ankur Handa, Tsung-Yi Lin, Gordon Wetzstein, Ming-Yu Liu, and Donglai Xiang. CoT-VLA: Visual

chain-of-thought reasoning for vision-language-action models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 1702–1713, 2025. 2

[28] Jinliang Zheng, Jianxiong Li, Zhihao Wang, Dongxiu Liu, Xirui Kang, Yuchun Feng, Yinan Zheng, Jiayin Zou, Yilun Chen, Jia Zeng, Ya-Qin Zhang, Jiangmiao Pang, Jingjing Liu, Tai Wang, and Xianyuan Zhan. X-VLA: Soft-prompted transformer as scalable cross-embodiment vision-language-action model. In *ICLR*, 2026. 1, 2, 4

[29] Ruijie Zheng, Yongyuan Liang, Shuaiyi Huang, Jianfeng Gao, Hal Daumé III, Andrey Kolobov, Furong Huang, and Jianwei Yang. TraceVLA: Visual trace prompting enhances spatial-temporal awareness for generalist robotic policies. In *International Conference on Learning Representations*, 2025. 2